

UNIVERSITY OF BUEA

REPUBLIC OF CAMEROON

FACULTY OF ENGINEERING
AND TECHNOLOGY

PEACE-WORK-FATHERLAND

DEPARTMENT OF COMPUTER
ENGINEERING



COURSE TITLE:

**INTERNET PROGRAMMING (J2EE) AND MOBILE
PROGRAMMING**

BridgeLearn

COURSE CODE: CEF 440

NAMES	MATRICULE
BERINYUY GILLES KILALON	FE23A024
ITIE LLOYD BONYEKI	FE23A069
NGUEFACK ELVIRA DIKIELLE	FE23A112
NKWAIN REMI KUMA	FE23A123
NKENGLEFAC DORENOEL	FE23A120

**COURSE INSTRUCTOR
DR. NKEMENI VALERY**

TASK 6: UI DESIGN AND IMPLEMENTATION

I- INTRODUCTION

BridgeLearn is an adaptive e-learning platform designed for students in sub-Saharan Africa who access education on unstable, high-cost mobile data connections. The platform addresses a problem established clearly in the requirements gathering phase of this project: 79% of surveyed students rated their internet stability at 3 out of 5 or below during live lessons, 74.6% had missed class or abandoned a platform due to data costs, and 92.5% access learning exclusively via mobile data. Standard e-learning platforms were not built for this context. BridgeLearn was.



The system is built on a two-frontend, one-backend architecture. Students access the platform through a mobile application that delivers course content adaptively — switching between video, audio-only, and live text transcript modes based on real-time bandwidth measurement. Instructors access the platform through a web dashboard where they create courses, upload content, build quizzes, and launch live sessions. Both frontends communicate with a single Node.js/Express backend connected to one database. A course published by an instructor is immediately visible to enrolled students. A quiz submitted by a student offline is queued locally and synced automatically when connectivity returns.

This report documents **Database Design and Implementation** of BridgeLearn. It covers the data elements the system needs to store, the conceptual design of those elements and their relationships, the entity-relationship diagram, the database schema implemented using Prisma ORM with SQLite, the backend implementation connecting the API to the database, and the integration that ties all three layers together into a working system.

II- DATA ELEMENTS

A data element is any piece of information the system needs to store, retrieve, or act on in order to fulfil its functional requirements. The following section identifies every data element in BridgeLearn, organised by the entity it belongs to, along with the attributes that describe it and the reason the system requires it.

User

The User entity is the foundation of the entire system. Every person who interacts with BridgeLearn, whether a student, instructor, or administrator is a user. The system needs to store a unique identifier, full name, email address, hashed password, role, preferred language, and account creation timestamp. The role attribute is the most critical — it determines what every other part of the system will allow that person to do. The language preference stores either English or French, satisfying the bilingual requirement established. Passwords are stored as bcrypt hashes, never in plain text.

Course

A Course is the primary unit of educational content in BridgeLearn. The system stores a unique identifier, title, description, publication status, the identifier of the instructor who created it, and timestamps for creation and last update. The publication status distinguishes

between courses that are still being prepared by the instructor and courses that are visible to students. A course cannot exist without an instructor — the instructor identifier is a required foreign key.

Enrolment

Enrolment resolves the many-to-many relationship between students and courses. A student can enrol in many courses, and a course can have many students. The system stores the student identifier, the course identifier, and the date of enrolment. This entity is what makes it possible to return only the courses a student is enrolled in, rather than every course in the system, when a student logs in.

Content

Content represents the learning materials within a course — videos, audio files, notes, PDFs, and transcripts. The system stores a unique identifier, the course it belongs to, a title, the content type, file paths for each available rendition (480p video, 240p video, audio-only MP3, transcript text), a processing status that tracks whether FFmpeg has finished transcoding, a downloadable flag indicating whether the content can be saved for offline access, a live flag distinguishing recorded content from live stream content, and the duration in seconds. The multiple file path attributes exist because BridgeLearn generates several renditions of every uploaded video — the adaptive player selects the appropriate one based on the student's bandwidth.

Quiz

A Quiz is an assessment attached to a course. The system stores a unique identifier, the course it belongs to, a title, the questions as a structured JSON array — each question containing its text, type, answer options, and correct answer — an optional deadline, total marks, and pass mark. Storing questions as JSON allows flexible question types (multiple choice, true/false) without requiring a separate questions table for this stage of the system.

Quiz Result

A Quiz Result records the outcome of a single student's attempt at a quiz. The system stores a unique identifier, the student identifier, the quiz identifier, the answers submitted as a JSON array, the calculated score, a boolean indicating whether the student passed, the submission timestamp, and a flag indicating whether the submission was made offline and synced later. The offline flag is essential for the integrity of the analytics — an instructor reviewing results needs to know which submissions were made under normal conditions and which were queued and synced.

Progress

The Progress entity tracks how far a student has advanced through a course and its individual content items. The system stores the student identifier, the course identifier, an optional content item identifier, a completion percentage, the timestamp of the last activity, and the total time spent in seconds. This entity feeds the progress bar displayed on the student's home dashboard and the analytics view available to the instructor.

Live Session

A Live Session represents a real-time lecture event created by an instructor and attended by students. The system stores a unique identifier, the course it belongs to, a socket room identifier used by the WebSocket server to route messages between the instructor and connected students, the start and end timestamps, the path to the recorded session file once the

session ends, and a participant count. The socket room identifier is the technical bridge between the REST API and the WebSocket layer — when a student joins a live session, they join the room identified by this value.

Streaming Session

A Streaming Session records the technical state of a single student watching a single content item. The system stores the student identifier, the content identifier, the most recently measured bandwidth in kilobits per second, latency in milliseconds, packet loss as a percentage, the current quality mode the player is operating in, total data consumed in megabytes, and the session start and end timestamps. This entity is written to every ten seconds during active playback and is the source of the QoE data that appears in the admin analytics dashboard.

QoE Log

The QoE Log is a detailed event-level record of quality changes during a session. Where the Streaming Session stores the current state, the QoE Log stores individual events — each row represents one ten-second interval. The system stores the student identifier, the content or live session identifier, the bandwidth, latency, and packet loss measured at that moment, the current quality mode, the number of buffering events in that interval, a Boolean indicating whether a mode switch occurred, a timestamp, and optionally a region derived from the student's connection. This granularity allows the admin to identify not just that students experienced quality problems, but exactly when, how often, and under what network conditions.

Notification

A Notification is an alert sent to a user about an event in the system. The system stores a unique identifier, the recipient user identifier, the notification type — new content, quiz deadline, or live session alert — a title, a message body, a read status Boolean, and a creation timestamp. Notifications are created by the backend when an instructor publishes content or starts a live session, and delivered to students via Firebase FCM push notifications on their devices.

Password Reset Token

The Password Reset Token supports the forgot password flow. The system stores a unique identifier, the user identifier, a randomly generated token string, an expiry timestamp, and a used Boolean. Once a token is used to reset a password it is marked as used and cannot be used again.

III- CONCEPTUAL DESIGN

BridgeLearn's database organises around two actors — students and instructors — both represented by a single User entity distinguished by a role attribute. Storing both in one table avoids duplicating shared attributes like name, email, and password. Language preference is stored at the account level so the system remembers a user's EN/FR choice across sessions.

A Course is created by one instructor and accessed by many students through an Enrolment entity, which resolves the many-to-many relationship between students and courses. A unique constraint prevents a student from enrolling in the same course twice. Deleting a course cascades to remove all related enrolments automatically.

Each Course contains Materials and Quizzes. A Material stores three video renditions — high quality, low quality, and audio-only — as a JSON structure, along with a timestamped transcript array. This supports the adaptive player's ability to switch between renditions based on bandwidth. A processing status field distinguishes content still being transcoded from content ready for access. A Quiz stores its questions as a JSON array and carries a published flag independent of the course's own published status, allowing an instructor to control their visibility separately.

When a student completes a quiz, a QuizSubmission is created recording their score, total questions, and individual answers. A LiveSession represents an active lecture, constrained to one per course at a time. It stores the current slide position and transcript text as fields the WebSocket server updates in real time. A Progress record tracks each student's completion rate and quizzes taken per course, updated in place on each access.

IV- ENTITY-RELATIONSHIP DIAGRAM

The entity-relationship diagram for BridgeLearn represents eight entities and the relationships between them. Each entity corresponds directly to a model in the Prisma schema, and each relationship reflects a real interaction between users, content, and learning activity in the system.

User is the central entity. Every person in BridgeLearn is a User, distinguished by a role attribute. A User who is a student has zero or more Enrolments and zero or more QuizSubmissions. A User who is an instructor creates Courses, though this relationship is stored as a plain string field on Course rather than a Prisma relation — a deliberate simplification discussed further in the database implementation section.

Course is the primary organisational unit. One Course has many Materials, many Quizzes, and many Enrolments. It is connected to a LiveSession when one is active, and its progress is tracked per student through the Progress entity. Both LiveSession and Progress reference a Course through plain string identifiers rather than enforced foreign key relations in Prisma, which means referential integrity for these two entities is maintained at the application layer rather than the database layer.

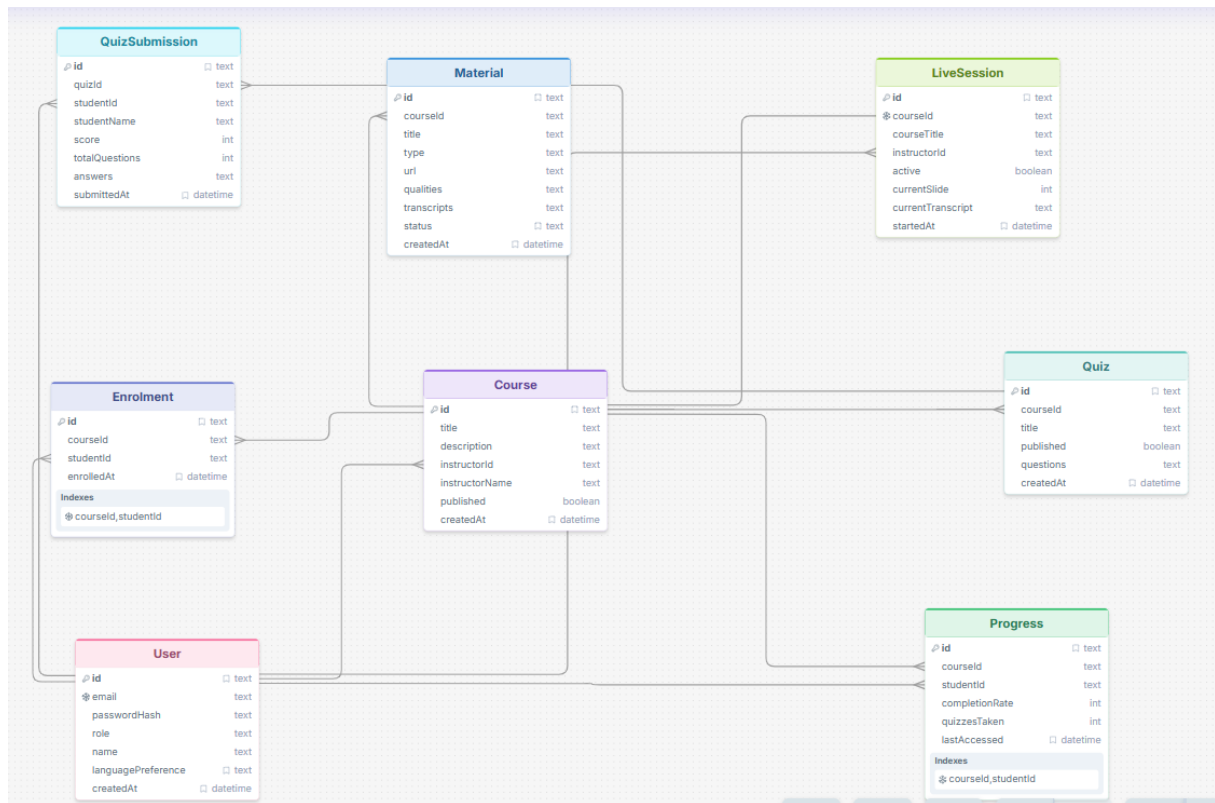
Enrolment resolves the many-to-many relationship between User and Course. One User can enrol in many Courses, and one Course can have many enrolled Users. A unique constraint on the combination of courseId and studentId prevents duplicate enrolments. Cascade deletion ensures that removing a Course or a User also removes all associated Enrolment records.

Material belongs to one Course and can only exist within that Course. Deleting a Course cascades to delete all its Materials.

Quiz belongs to one Course and has zero or more QuizSubmissions. One QuizSubmission belongs to exactly one Quiz and one User. Cascade deletion applies on both relations — removing a Quiz removes its submissions, and removing a User removes their submissions.

LiveSession is constrained to one per Course at a time through a unique constraint on the courseId field. It holds mutable state — current slide, current transcript — that the WebSocket server updates in real time during an active lecture.

Progress holds one record per student per course, enforced through a unique constraint on the combination of courseId and studentId. It is updated in place on each student interaction rather than creating new records over time.



V- DATABASE IMPLEMENTATION

BridgeLearn's database is implemented using SQLite as the database engine and Prisma as the object-relational mapping layer. SQLite was chosen for this stage of development because it runs as an embedded process within the Node.js server with zero additional configuration — there is no separate database server to install, start, or manage. The entire database is a single file on disk, which makes the development environment portable and reproducible across team members. Prisma provides a typed query interface over SQLite, abstracting raw SQL while still generating efficient queries. When the system is ready for production deployment, migrating from SQLite to PostgreSQL requires changing a single line in the Prisma schema the datasource provider — and rerunning the migration command.

The schema defines eight models: User, Course, Enrolment, Material, Quiz, QuizSubmission, LiveSession, and Progress. Each model maps directly to a database table. Primary keys are UUID strings generated at the application layer using Prisma's `uuid()` default, which ensures globally unique identifiers that do not expose sequential row counts to clients.

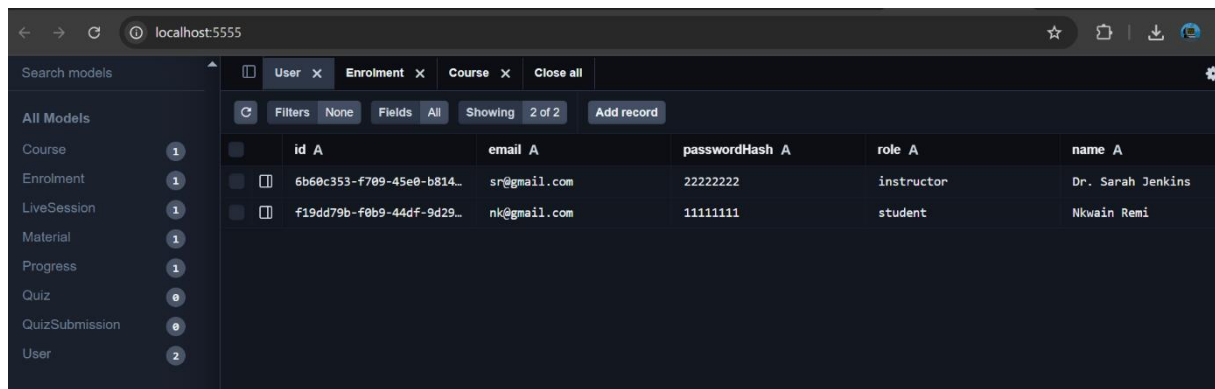


Figure 1 Prisma Studio View of Database

Three categories of constraints are enforced at the database level. Unique constraints prevent duplicate records where the business logic demands it — a student cannot enrol in the same course twice, a course cannot have more than one active live session, and a student cannot have more than one progress record per course. Cascade deletion rules ensure referential integrity when parent records are removed — deleting a Course automatically removes all its Materials, Quizzes, and Enrolments, and deleting a User removes all their Enrolments and QuizSubmissions. Default values are set at the schema level for fields like `published`, `active`, `completionRate`, and `languagePreference`, ensuring that new records are created in a consistent state without requiring the application to supply these values explicitly.

Two data types in the schema deserve specific attention. The `questions` field on `Quiz` and the `answers` field on `QuizSubmission` are stored as JSON strings rather than in normalised relational tables. This is a deliberate trade-off — quiz questions have a variable structure depending on question type, and normalising them into a separate table would add join complexity without meaningful benefit at this scale. The `qualities` field on `Material` similarly stores the three HLS rendition paths as a JSON object. These fields are serialised before writing and deserialised after reading at the application layer.

The database is initialised and kept in sync with the schema through Prisma Migrate. Running `npx prisma migrate dev` reads the current schema file, compares it against the existing database state, generates a SQL migration file describing the differences, and applies it. This migration history is committed to the repository alongside the schema, giving the team a complete record of every structural change made to the database over the course of the project.

VI- BACKEND IMPLEMENTATION

BridgeLearn's backend is a Node.js server built with the Express framework. It serves as the single point of communication for both frontends — the instructor web dashboard and the student application — handling all REST API requests and real-time WebSocket events through one running process. The backend is written in TypeScript, which provides type safety across route handlers, middleware, and database queries, catching errors at compile time rather than at runtime.

The server initialises in a single entry point file that configures middleware, registers route handlers, and starts both the HTTP server and the WebSocket server on the same port. CORS is configured to accept requests from the two frontend origins, and the request body is parsed as JSON for all incoming requests.

Authentication is handled using JSON Web Tokens. When a user logs in with a valid email and password, the server verifies the password against the stored bcrypt hash and, if correct, signs a JWT containing the user's id, name, email, and role. This token is returned to the client and attached to the Authorization header of every subsequent request. A middleware function verifies the token on every protected route before the request reaches the route handler. Routes that are restricted to instructors check the role field in the decoded token and reject requests from students with a 403 response.

```
NEWBORN COMPUTER@BUDDY-77 MINGW64 ~/Downloads/BridgeLearn-version1/backend
Ⓢ $ npm run dev

> adaptive-elearning-backend@1.0.0 dev
> nodemon src/index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/index.js`
E-Learning monorepo server active on port 3000
WebSocket Authenticated: Nkwain Remi (student)
WebSocket Authenticated: Nkwain Remi (student)
```

Figure 2 WebSocket Authentication shown in our terminal

The API is organised into route groups corresponding to the main entities in the system. The authentication routes handle registration and login. The course routes handle creation, retrieval, publishing, and deletion of courses, with write operations restricted to instructors and read operations available to authenticated students. The material routes handle content upload and retrieval, including the logic that processes uploaded video files into multiple quality renditions. The quiz routes handle creation and publishing by instructors, and submission by students. The enrolment routes allow students to enrol in published courses. The progress routes read and update a student's completion rate per course. The live session routes allow an instructor to start and end a session, and students to retrieve the current session state for a course they are enrolled in.

Real-time communication for live lectures is handled by a WebSocket server running alongside Express on the same port. When an instructor starts a live session, connected students receive an immediate notification through the WebSocket connection. As the instructor advances slides or the transcript updates, the server broadcasts the new state to all connected clients in real time. The student application maintains a persistent WebSocket connection during a live session and updates its display whenever a broadcast is received. When the instructor ends the session, a final broadcast notifies all connected students that the session has concluded.

Error handling is consistent across all routes. Unhandled errors are caught by a global error handling middleware that returns a structured JSON response with an appropriate HTTP status code and message. Validation errors return 400, authentication failures return 401, authorisation failures return 403, and missing resources return 404. This consistency means both frontends can handle errors predictably without needing to parse different error formats from different routes.

VII- CONNECTING THE DATABASE TO THE BACKEND

The connection between BridgeLearn's database and its backend is managed entirely through the Prisma Client. At startup, the backend instantiates a single PrismaClient object which establishes and maintains a connection pool to the SQLite database file. This instance is created once and exported as a shared module, meaning every route handler across the entire application imports and uses the same client rather than creating new connections per request. This pattern keeps connection overhead minimal and prevents the database file from being locked by multiple competing handles simultaneously, which is a real concern with SQLite under concurrent load.

The database file location is not hardcoded into the application. It is read from the `DATABASE_URL` environment variable defined in the `.env` file at the root of the project. In development this points to a local file `file:./dev.db` —sitting alongside the source code. This means the entire database can be reset by deleting that file and rerunning the migration command, which proved useful during development when the schema changed between iterations.

The Prisma schema file serves as the contract between the database and the backend. Any change to the data model — adding a field, introducing a new relation, changing a constraint — is made first in the schema file and then propagated to the database by running `npm run prisma migrate dev`. This command generates a timestamped SQL migration file recording exactly what changed, applies it to the database, and regenerates the Prisma Client so the TypeScript types immediately reflect the new structure. The backend cannot compile with stale types, which means a schema change that is not migrated will surface as a type error before the server starts rather than as a runtime failure.

A concrete example of the full connection in action is the quiz submission flow. When a student submits a quiz, the route handler receives the answers in the request body, queries the Quiz record using the Prisma Client to retrieve the correct answers, calculates the score, and writes a new QuizSubmission record in a single transaction. The instructor's dashboard then queries QuizSubmission records for that quiz through the same Prisma Client and displays the results. The same database connection, the same client instance, and the same underlying SQLite file serve both sides of that interaction — which is the practical demonstration of the one-backend architecture that BridgeLearn is built on.

VIII- CONCLUSION

Task 6 established the data foundation that BridgeLearn operates on. Beginning from the functional requirements defined in Task 2, the team identified the data elements the system needs to store, designed their relationships conceptually, expressed those relationships as an

entity-relationship diagram, and implemented them as a live SQLite database through Prisma ORM. The backend connects to that database through a single shared Prisma Client instance, serving both the instructor dashboard and the student application from one consistent data source.

The database as implemented supports every core user journey in the system — course creation and publication, student enrolment, adaptive content delivery, quiz submission with offline sync, live session broadcasting, and progress tracking. What remains is validating that those journeys perform correctly end to end under the network conditions BridgeLearn was designed for, which is the work of the testing and validation phase that follows.

REFERENCES:

- DrawSQL: <https://drawsql.app/draw>
- GeekforGeeks: <https://www.geeksforgeeks.org/blogs/backend-development/>